ROLL NO: | | | | | | | | | | |

(To fill the roll no is compulsory)

INVERTIS UNIVERSITY BAREILLY

EVEN SEMESTER EXAMINATION - 2024-25

B.Tech. (CE/EE/ME/CSE)

Semester: II

SUBJECT CODE: BCS201

PROBLEM SOLVING IN C

Time: 03:00 Hrs. **Maximum Marks: 50**

NOTE: THE QUESTION PAPER CONTAINS THREE SECTIONS. ATTEMPT ALL THE SECTIONS.

SECTION – A

NOTE: Attempt all parts of the following question. **(10 Marks)**

Q. 1. Answer the following questions: **(1 × 5 = 5 Marks)**

(i) What is a wild pointer in C?

(ii) How does fopen() function work in file handling?

(iii) What is the difference between malloc() and calloc()?

(iv) What is the purpose of macros in C?

(v) What is a thread in C?

Q. 2. Answer the following questions: **(1 × 5 = 5 Marks)**

(i) What is the difference between structure and union?

(ii) What is the role of ftell() and fseek() in file handling?

(iii) What is memory fragmentation in dynamic memory allocation?

(iv) What are header files in C pre-processor?

(v) What is the purpose of pthread_join() in multithreading?

SECTION – B

NOTE:Attempt all questions. **(5×2×2 = 20 Marks)**

Q.3.(i)(a) Write about pointer to pointer with an example.

OR

(b) What is circular linked list? How is it different from a singly linked list?

(ii)(a) Explain file handling modes in C (r, w, a, rb, wb).

OR

(b) How does fscanf() and fprintf() work? Explain with an example.

Q.4.(i)(a) Discuss mutex locks and their role in multithreading.

OR

(b) What are race conditions in multithreading? How can they be prevented?

(ii)(a) Describe the implementation of AVL tree with an example of program.

OR

(**b**) Explain open addressing and chaining methods in hashing.

SECTION – C

NOTE: Attempt all questions. **(10 × 2 = 20 Marks)**

Q.5.(a) Discuss in detail the macro substitution process with an example.

OR

(b) What is TCP socket programming? Explain with a client-server model.

Q.6.(a) Write a program to implement a singly linked list in C with insertion and deletion.

OR

(b) Explain memory management in C with malloc(), calloc(), realloc(), and free().